

ActuarialBench

Reproducible Benchmark of Frontier LLM Reliability in Actuarial Science

Amine Manai

Experiment 20260826T172307Z_c8d36e05

Abstract

We evaluated 4 externally asserted language-model routes on 48 deterministic actuarial tasks with 3 repetitions, yielding 576 planned calls. Ground truth was computed independently, numerical and code outputs were graded mechanically, and validation tasks used exact planted defect codes. claude-opus-5 achieved the highest capability mean (0.876), while gpt-5.6-sol produced the most exact-correct calls. The leading capability means were not statistically separated after multiplicity correction, so the experiment supports metric- and domain-specific findings rather than a universal model ranking.

Contents

1	Introduction	2
2	Research Questions	2
3	Benchmark Design	3
3.1	Experimental unit and task portfolio	3
3.2	Route configuration	3
4	Actuarial Task Construction	3
4.1	Generated numerical and component tasks	3
4.2	Coding tasks	4
4.3	Validation and missing-information tasks	4
5	Ground Truth and Grading	4
6	Experimental Controls	4
7	Statistical Methodology	5
7.1	Capability and operational views	5
7.2	Uncertainty and paired comparisons	5
7.3	Reliability and calibration	5
8	Results	5
8.1	Overall capability and exact correctness	5
8.2	Domain and task-format results	6
8.3	Paired evidence	7
9	Failure Analysis and Reliability	9

9.1	Failure taxonomy	9
9.2	Validation and missing-information behavior	10
9.3	Repeatability, schema adherence, and confidence	11
10	Cost and Latency	12
11	Limitations and Threats to Validity	14
12	Conclusion	14
A	Reproducibility Checklist	15
B	Artifact Map	15

1 Introduction

An actuarial answer can be fluent and still fail at the point where the work matters: an assumption may be invented, a reserve formula may be misapplied, a probability may be impossible, or generated code may not reproduce the stated calculation. ActuarialBench measures these observable failures under a controlled protocol. It is a compact empirical study, not a conversational demonstration or a subjective writing contest.

The benchmark’s primary contribution is an auditable path from deterministic task generation to preserved model output, objective grading, paired statistical analysis, and a report generated from machine-readable artifacts. The study asks how reliably four black-box language-model routes solve and validate actuarial tasks, where their failure patterns differ, and whether measured differences are large enough to survive uncertainty and multiple-comparison controls.

The analysis deliberately keeps capability, route availability, calibration, latency, and cost separate. A route can score well when it returns a usable answer yet fail more often at the API or schema boundary. Combining those dimensions into one post-hoc score would conceal the operational trade-off and introduce discretionary weights after observing the results.

2 Research Questions

The experiment addresses the following questions under the frozen task bank and provider configuration:

1. Which route achieves the highest objective actuarial score and exact-correct rate?
2. Which route produces the most reliable executable actuarial code?
3. Which route detects planted validation defects and missing assumptions most accurately?
4. Do any routes introduce unsupported issue codes on missing-information tasks?
5. How stable are scores across three repeated calls to the same generated task?
6. How often do schema, timeout, or provider failures prevent capability assessment?
7. Does reported confidence correspond to exact correctness?
8. How do results vary across actuarial domains and task formats?
9. Are pairwise differences statistically supported after multiplicity correction?
10. What accuracy-latency and accuracy-cost trade-offs are observable from provider-reported metadata?

No question asks which model is universally best. Every conclusion remains conditional on this task portfolio, prompt contract, route configuration, output budget, and experiment date.

3 Benchmark Design

3.1 Experimental unit and task portfolio

The full bank contains 48 deterministic tasks. Six generated instances cover each of life contingencies, non-life pricing, reserving, risk, statistical modelling, and survival analysis. Validation contributes 12 tasks: six contain planted pricing defects and six withhold assumptions required for a life-insurance calculation. Four routes answered every task three times, creating 576 planned route-task-repetition cells.

Table 1: Recorded experiment provenance. Values are read from the frozen manifest.

Field	Recorded value
Experiment	20260826T172307Z_c8d36e05
Benchmark version	0.1.0
Git commit at execution	fcefdd33c30e1ed88c6ee4bc64921fee2b7e668b
Created (UTC)	2026-08-26T17:23:07.573182+00:00
Tasks	48
Routes	4
Repetitions per task and route	3
Planned calls	576

The task portfolio is intentionally balanced by generated family, not calibrated to the workload of any particular insurer, qualification syllabus, or national market. Domain means therefore describe performance within this portfolio; they are not prevalence-weighted estimates of production actuarial work.

3.2 Route configuration

The configured labels were `gpt-5.6-sol`, `deepseek-v4-flash`, `claude-opus-5`, and `glm-5.3`. All were accessed through AgentRouter. Route identities are provider assertions and were not independently verified. The OpenAI Responses transport was used for GPT and DeepSeek, the Anthropic Messages transport for Claude, and OpenAI Chat Completions for GLM. Claude preserved separate system and user messages; compatible routes received the same content through their supported prompt transport.

Each request used a fresh context and a 2,000-token output budget. DeepSeek and GLM used the configured low reasoning effort. Requests ran concurrently across routes but sequentially within each route, so no route key had more than one call in flight. The manifest records route metadata, prompt and task hashes, random seeds, retry policy, timestamps, and the Git commit used for execution.

4 Actuarial Task Construction

4.1 Generated numerical and component tasks

Fixed-seed generators produced novel parameter combinations while retaining analytic ground truth. Life tasks multiply age-specific annual survival factors. Pricing tasks compute frequency, severity, pure premium, and aggregate variance under a compound model. Risk tasks evaluate lognormal value-at-risk and tail value-at-risk components. Statistical tasks recover exponential maximum-likelihood quantities, while survival tasks reconstruct Kaplan-Meier components.

Multi-component scoring gives credit only for the components that meet their predefined tolerances. This avoids treating an answer with one correct intermediate value as either wholly correct or wholly

wrong. Exact correctness still requires full credit on every required component.

4.2 Coding tasks

Reserving tasks require a Python function for Chain Ladder calculations. Generated code is parsed with an abstract-syntax-tree allowlist, limited to one required function, and executed in an isolated Python subprocess with restricted built-ins and a short timeout. Hidden cases check both execution and actuarial output. This mechanism reduces accidental host access but is not a hardened security sandbox.

4.3 Validation and missing-information tasks

The planted-defect tasks encode exact issue identifiers for negative exposure, overdispersion, future-data leakage, and impossible probabilities. Missing-information tasks require the route to identify absent mortality and interest assumptions without adding unsupported issue codes. Precision, recall, F1, and critical-defect recall follow directly from the expected and reported issue-code sets.

This design makes validation grading reproducible, but it also narrows what can be claimed. The benchmark measures recognition of the injected defect taxonomy. It does not grade the full quality of an actuarial model-validation memorandum or prove that a route would detect novel defects outside the frozen set.

5 Ground Truth and Grading

Reference answers are computed independently of model output using deterministic Python and actuarial formulas. Each task stores its seed, expected values, component tolerances, metadata, required concepts, and grading policy. Unit tests exercise generator invariants such as bounded probabilities, finite reserves, non-negative risk measures, and reproducible task hashes.

Numerical values use task-specific absolute and relative tolerances through `numpy.isclose`. Coding answers receive separate execution and hidden-case component scores. Diagnosis answers use exact set comparisons. The parser accepts a JSON object and tolerates surrounding code fences or recoverable surrounding text, but it records schema violations rather than silently inventing missing fields.

The primary score is the arithmetic mean of objective component scores. *Exact correct* means a score of one. Explanation checks are recorded as secondary evidence and do not replace mathematical or code correctness. Provider and timeout failures remain tagged separately from model-content errors.

6 Experimental Controls

Every route received the same task text, system instructions, task order, maximum output budget, grading rules, and reference answers. Each task began with a fresh context. Repetition identifiers were preserved rather than averaging or selecting the best response. Raw text, parsed output, component scores, latency, token counts, finish reason, route metadata, and error state were written for every call.

The lowest-common-denominator policy cannot make heterogeneous APIs identical. Prompt transport differs for Claude, and provider-side routing or hidden model updates may differ across labels. Those deviations are recorded instead of being treated as controlled model properties. Provider-reported cost was accepted only when supplied; no public price list was retrofitted to calls after execution.

The experiment directory is append-only in normal operation. Its core artifacts are `manifest.json`, `responses.jsonl`, `scores.jsonl`, and `analysis.json`. The published run contains all 576 planned cells without duplicates or omissions.

7 Statistical Methodology

7.1 Capability and operational views

Two views prevent route failures from being mistaken for mathematical errors. The all-call mean retains every planned call and assigns the recorded zero score to API or timeout failures. The capability mean excludes those failures and estimates performance conditional on receiving a gradeable route response. Both are reported with their denominators.

Schema failures remain in the capability view because the route returned content but did not satisfy the common output contract. Schema-failure rates use capability observations as their denominator. Calibration metrics use capability rows with a reported confidence value, so they should not be read as unconditional calibration of unavailable or unparsable responses.

7.2 Uncertainty and paired comparisons

Bootstrap 95% intervals use 2,000 fixed-seed resamples of observed scores. Pairwise comparisons align routes on task and repetition and exclude a cell when either route has an API or timeout failure. The paired sign-permutation test evaluates the mean score difference, while paired Cohen’s d_z describes standardized effect size. Exact McNemar tests compare discordant exact-correct outcomes on the same aligned cells.

Six pairwise route comparisons create a multiple-testing problem. Holm adjustment is applied separately to permutation and McNemar p -values. A descriptive lead is called statistically supported only when the point estimate favors the route, the interval excludes zero, and the adjusted test supports a difference. This rule prevents small ranking changes from becoming unsupported winner claims.

7.3 Reliability and calibration

Repeat agreement is the share of tasks whose available repetition scores are identical. Mean within-task standard deviation measures the magnitude of repeat variation without confounding it with different task difficulty. The Brier score compares confidence, scaled to $[0, 1]$, with exact correctness. Five equal-width confidence bins support the reliability plot; sparse bins are shown only when they contain observations.

8 Results

8.1 Overall capability and exact correctness

Across capability observations, claude-opus-5 recorded the highest mean partial-credit score (0.876), followed by gpt-5.6-sol (0.845). The paired difference was 0.034 with a bootstrap 95% interval of $[-0.034, 0.105]$ and Holm-adjusted permutation $p = 0.363$; the separation was not statistically supported. gpt-5.6-sol produced the most exact-correct calls (119 of 144). glm-5.3 had the lowest median latency at 6.55 seconds. These are different operating dimensions and are not combined into a composite ranking.

Table 2: Overall performance. Capability metrics exclude API and timeout failures; all-call metrics retain them.

Route	Calls	Capability n	All-call mean	Capability mean [95% CI]	Exact correct	API fail
claude-opus-5	144	141	0.858	0.876 [0.826, 0.922]	116/144	2.1%
gpt-5.6-sol	144	144	0.845	0.845 [0.786, 0.899]	119/144	0.0%
deepseek-v4-flash	144	144	0.630	0.630 [0.554, 0.708]	89/144	0.0%
glm-5.3	144	130	0.516	0.571 [0.488, 0.650]	60/144	9.7%

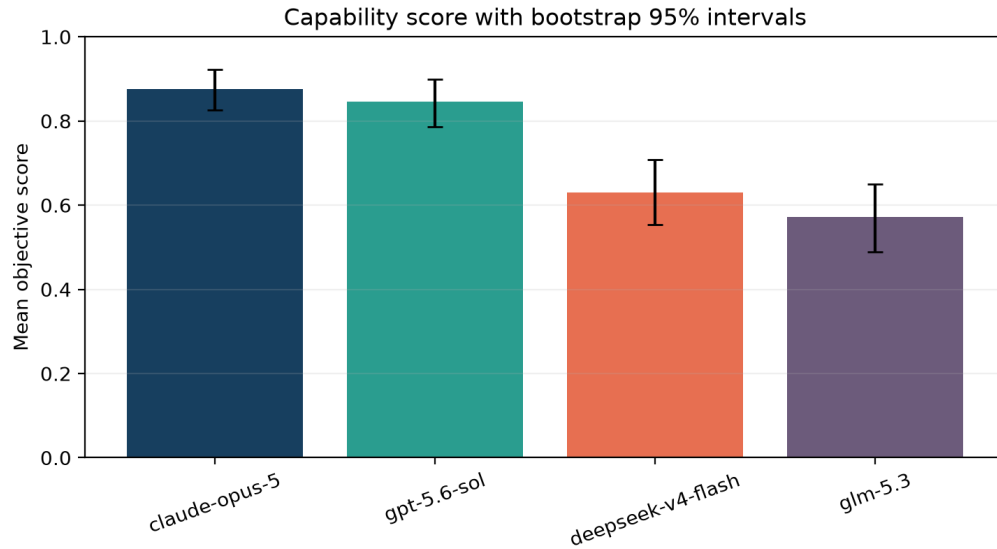


Figure 1: Mean capability score with fixed-seed bootstrap intervals. Overlap between the leading intervals is resolved through the paired tests rather than visual ranking alone.

The leading routes differ depending on the outcome definition. Claude had the highest capability partial-credit mean, while GPT produced more exact-correct calls and had no API failures. GLM’s all-call result fell below its capability mean because no-text route responses were retained as operational failures. This distinction is material: conditional mathematical performance and end-to-end route reliability answer different questions.

8.2 Domain and task-format results

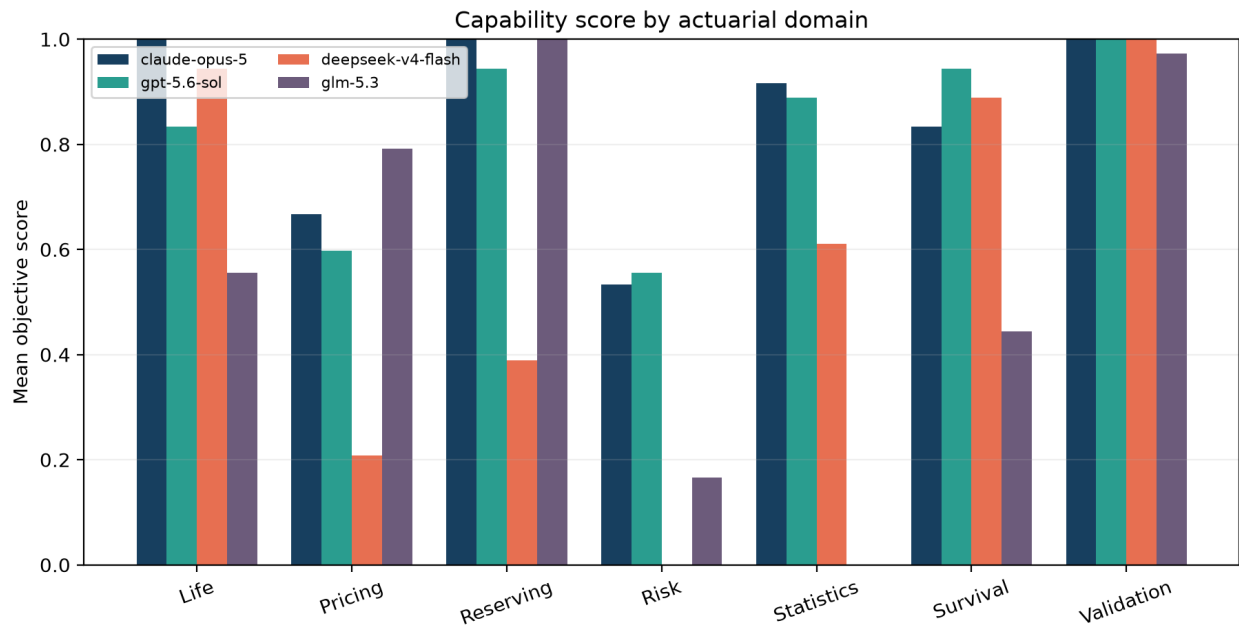


Figure 2: Capability score by domain. Each domain contains six generated tasks except validation, which contains twelve.

Table 3: Mean capability score by actuarial domain.

Route	Life	Pricing	Reserving	Risk	Statistics	Survival	Validation
claude-opus-5	1.000	0.667	1.000	0.533	0.917	0.833	1.000
gpt-5.6-sol	0.833	0.597	0.944	0.556	0.889	0.944	1.000
deepseek-v4-flash	0.944	0.208	0.389	0.000	0.611	0.889	1.000
glm-5.3	0.556	0.792	1.000	0.167	0.000	0.444	0.972

No route dominated every domain. Claude led life and statistics and shared the top reserving result with GLM. GPT led risk and survival, while GLM led pricing. GPT, Claude, and DeepSeek all reached the ceiling on validation; GLM was close to it. DeepSeek’s aggregate result was constrained mainly by pricing, risk, and coding rather than validation or life tasks.

Table 4: Mean capability score by objective task format. Diagnosis is the average of precision, recall, F1, and critical recall.

Route	Coding	Diagnosis	Multi	Numerical
claude-opus-5	1.000	1.000	0.746	1.000
gpt-5.6-sol	0.944	1.000	0.747	0.833
deepseek-v4-flash	0.389	1.000	0.427	0.944
glm-5.3	1.000	0.972	0.351	0.556

The format view explains part of the domain pattern. Claude completed every gradeable coding task correctly. GPT was close on coding and strong across multi-component tasks. DeepSeek remained competitive on single numerical answers but lost substantial credit on multi-component and coding tasks. GLM’s gradeable coding responses were correct, yet route failures clustered on that family and are visible only in the all-call and API-failure measures.

8.3 Paired evidence

Table 5: Complete paired comparisons. Differences are left minus right. Holm adjustments are applied separately to the permutation and McNemar test families.

Comparison	Difference [95% CI]	d_z	Holm p	McNemar	Holm p	Pairs
claude-opus-5 vs deepseek-v4-flash	0.232 [0.158, 0.312]	0.496	0.003		< 0.001	141
claude-opus-5 vs glm-5.3	0.278 [0.189, 0.362]	0.565	0.003		< 0.001	127
claude-opus-5 vs gpt-5.6-sol	0.034 [-0.034, 0.105]	0.080	0.363		1.000	141
deepseek-v4-flash vs glm-5.3	0.088 [-0.002, 0.185]	0.165	0.119		< 0.001	130
deepseek-v4-flash vs gpt-5.6-sol	-0.215 [-0.292, -0.137]	-0.426	0.003		< 0.001	144
glm-5.3 vs gpt-5.6-sol	-0.265 [-0.358, -0.175]	-0.507	0.003		< 0.001	130

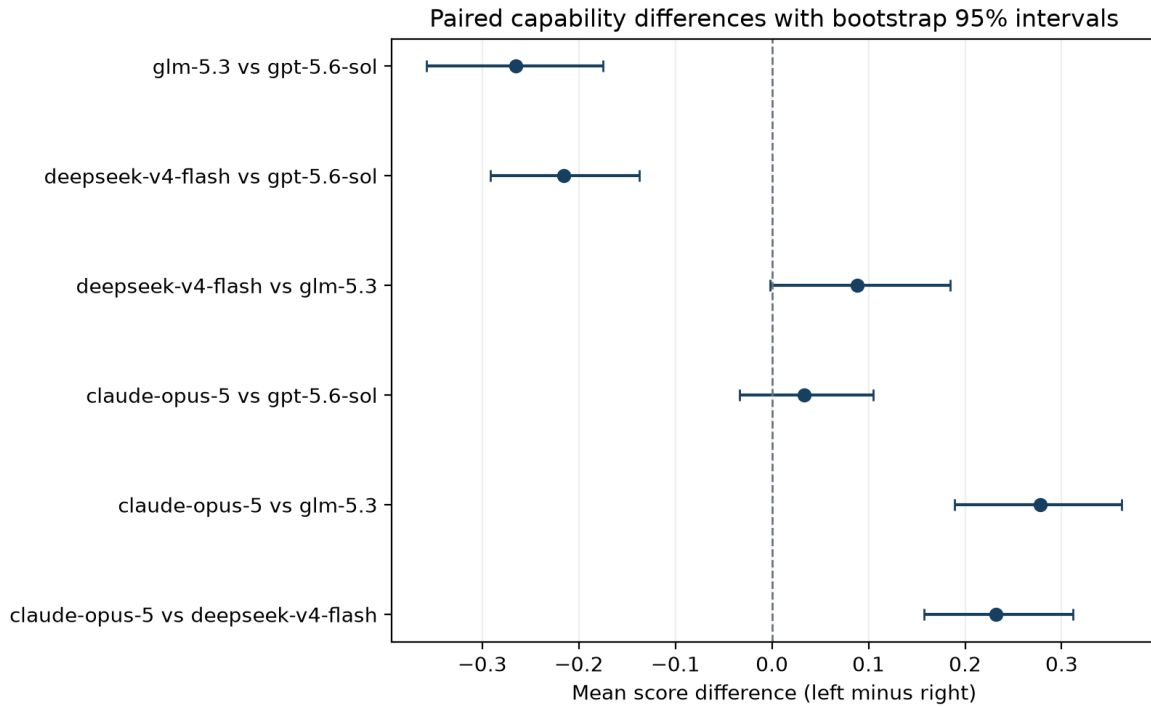


Figure 3: Paired capability-score differences. Positive values favor the route named on the left.

Claude and GPT were not statistically separated on mean capability score. Both showed statistically supported advantages over DeepSeek and GLM in the paired continuous-score analysis. DeepSeek’s descriptive lead over GLM was not supported after Holm correction, even though their exact-correct outcomes produced a significant McNemar result. That disagreement reflects different estimands: partial-credit magnitude versus discordant all-or-nothing correctness. Pair counts vary because a capability comparison requires a gradeable observation from both routes in the same task-repetition cell.

9 Failure Analysis and Reliability

9.1 Failure taxonomy

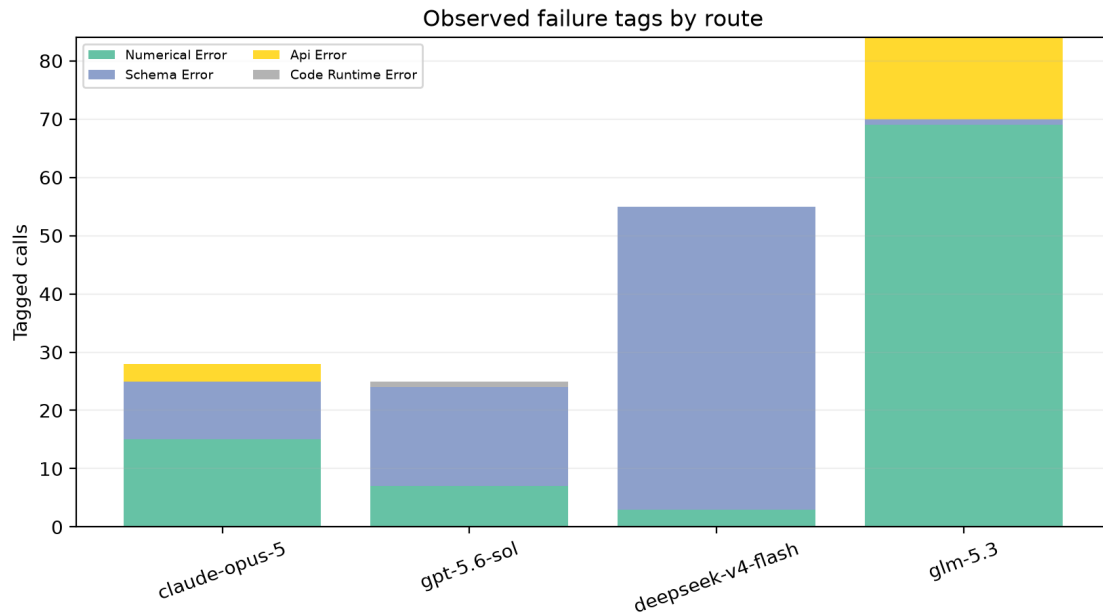


Figure 4: Failure tags recorded across all calls. One call may carry more than one tag.

Numerical and schema errors accounted for nearly all content-related failure tags. API errors were limited to no-text responses from Claude and GLM, and one coding response produced a runtime error. Failure tags are diagnostic labels, not mutually exclusive outcome classes, so their counts should not be summed into an error rate.

Table 6: Observed failure-tag counts by route.

Failure tag	claude-opus-5	gpt-5.6-sol	deepseek-v4-flash	glm-5.3
Api Error	3	0	0	14
Code Runtime Error	0	1	0	0
Numerical Error	15	7	3	69
Schema Error	10	17	52	1

9.2 Validation and missing-information behavior

Table 7: Validation performance on gradeable diagnosis responses. The unsupported issue rate is one minus issue-code precision on missing-information tasks.

Route	Precision	Recall	F1	Critical recall	Unsupported issue rate
claude-opus-5	1.000	1.000	1.000	1.000	0.0%
gpt-5.6-sol	1.000	1.000	1.000	1.000	0.0%
deepseek-v4-flash	1.000	1.000	1.000	1.000	0.0%
glm-5.3	1.000	1.000	1.000	1.000	0.0%

All four routes detected the expected validation issues without unsupported issue codes on their gradeable responses. The zero critical-miss and unsupported-issue rates are favorable observations, but they also reveal a ceiling effect. This validation subset did not discriminate among the routes and cannot establish comparative superiority. Harder mixtures of subtle defects, plausible distractors, and partially missing assumptions are needed before making stronger model-risk claims.

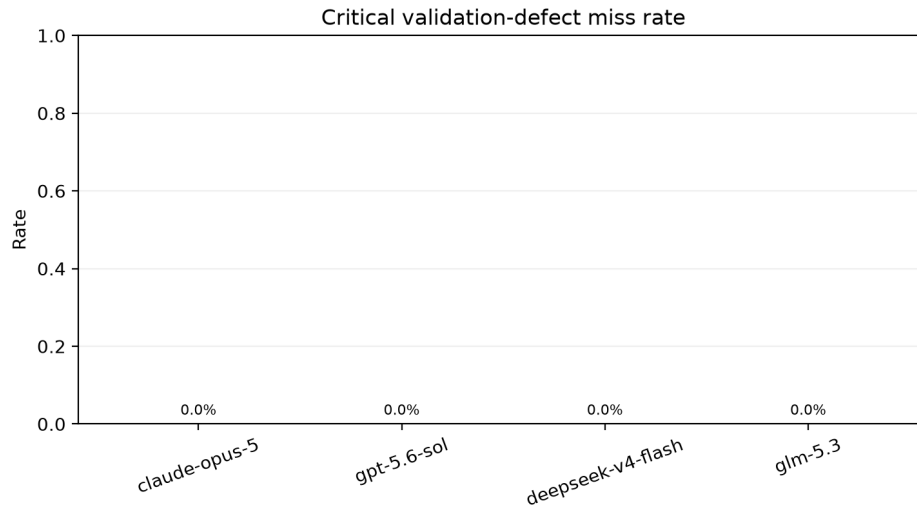


Figure 5: Critical validation-defect miss rate on gradeable diagnosis responses. Every observed rate was zero.

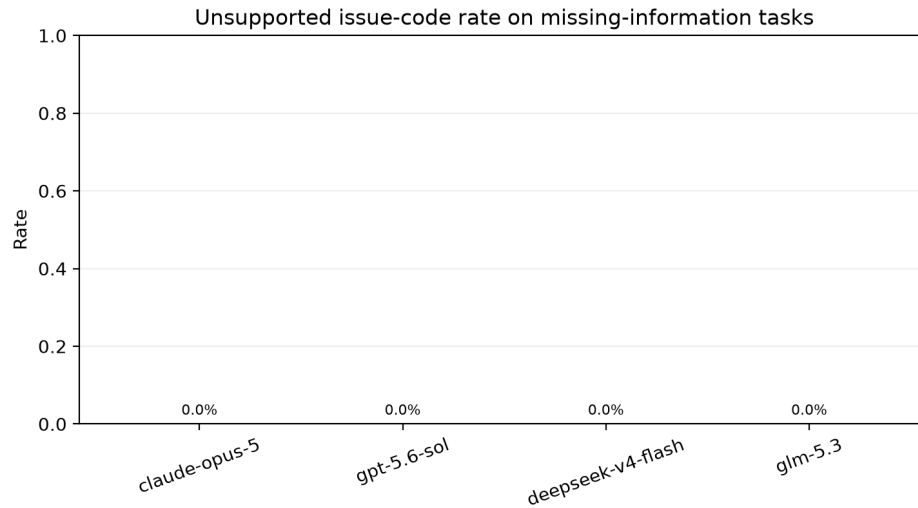


Figure 6: Unsupported issue-code rate on missing-information tasks. This is a conservative code-level proxy, not a semantic audit of every sentence.

9.3 Repeatability, schema adherence, and confidence

Table 8: Operational reliability and calibration. Brier scores are conditional on a reported confidence value; lower is better.

Route	Schema fail	Repeat agreement	Within-task SD	Brier	Median latency	Median output tok.
claude-opus-5	7.1%	74.5%	0.084	0.103	17.44 s	910
gpt-5.6-sol	11.8%	66.7%	0.149	0.062	16.51 s	480
deepseek-v4-flash	36.1%	75.0%	0.111	0.033	13.04 s	1426
glm-5.3	0.8%	62.8%	0.114	0.489	6.55 s	396

DeepSeek had the highest repeat-agreement rate by a narrow margin, but its schema-failure rate was substantially higher. GPT varied more in score across repeated calls while producing no API failures. GLM returned schema-conforming content when text was present, yet its route-level no-text failures reduced usable coverage. These measures should be read together: consistency among returned answers does not compensate for unavailable answers, and schema adherence does not establish mathematical correctness.

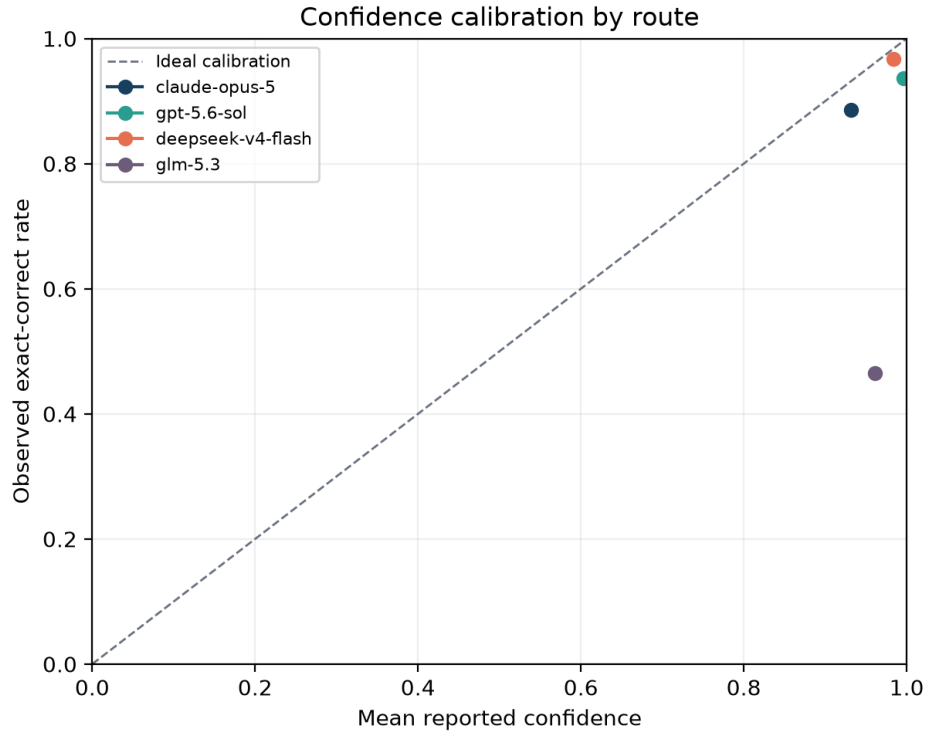


Figure 7: Confidence calibration for capability observations with reported confidence. Points aggregate observations into five equal-width bins.

The calibration curves show substantial route-specific differences, particularly in high-confidence bins. Because confidence may be absent when parsing fails, the Brier scores are conditional and can favor a route whose least usable responses are excluded. They are useful diagnostics under the recorded contract, not complete measures of epistemic calibration.

10 Cost and Latency

No route returned provider-reported cost for this experiment. Cost per task, cost per correct task, and an accuracy–cost Pareto comparison therefore remain unknown; no pricing estimate was substituted.

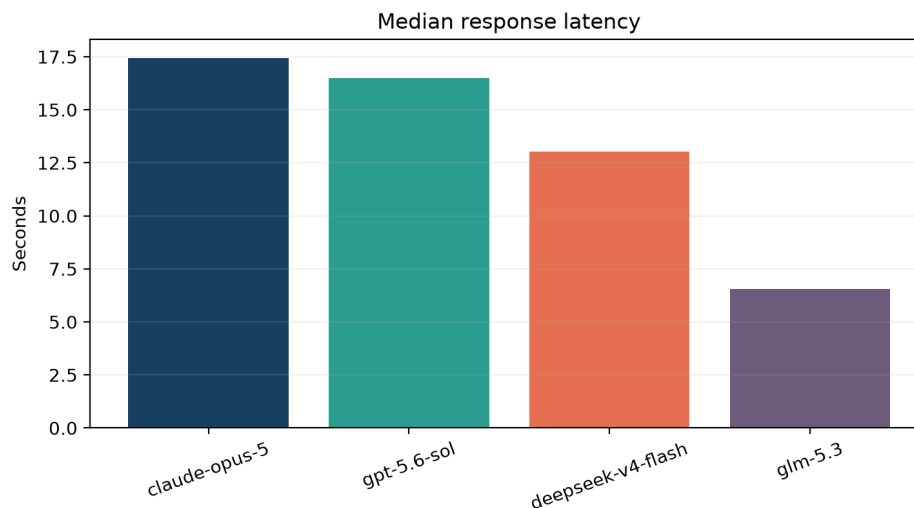


Figure 8: Median end-to-end response latency across recorded calls. Concurrency across routes means these values reflect the observed provider conditions, not a controlled throughput benchmark.

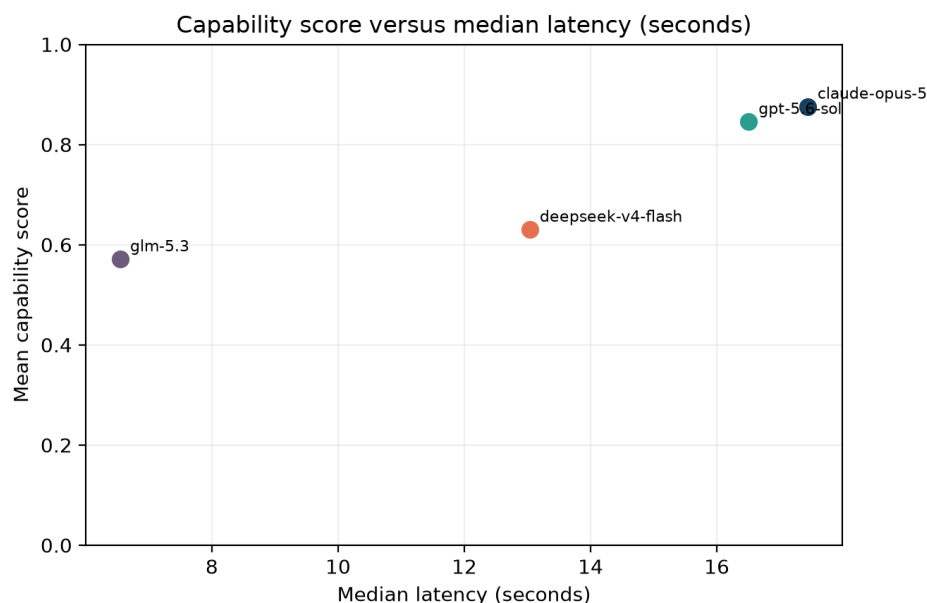


Figure 9: Observed capability-latency trade-off. Lower latency and higher score are preferable, but four provider-routed points are insufficient for a general efficiency frontier.

GLM was the fastest observed route, while Claude and GPT occupied the higher-capability, higher-latency region. No architecture-level explanation follows from this pattern because network conditions, provider routing, output length, and hidden serving configuration were not independently controlled. Cost comparisons remain unavailable rather than being reconstructed from public list prices that may not match AgentRouter billing.

11 Limitations and Threats to Validity

Construct validity. The bank measures objective numerical, component, coding, and issue-code outcomes. It does not cover regulatory writing, expert judgement under ambiguous policy wording, long-horizon workflow execution, or human review quality. The validation subset reached a ceiling and should be expanded before comparative claims are made in that domain.

External validity. Forty-eight generated tasks form a useful portfolio study but not a representative sample of all actuarial work. Six tasks within most domains are parameter variants of one family, so domain breadth is narrower than the task count alone suggests. Results should not be generalized to a different jurisdiction, provider, prompt, tool environment, or model version.

Route identity and serving conditions. AgentRouter controls routing and asserts the model labels. The study cannot verify underlying weights, quantization, system prompts, batching, or provider updates. Latency and no-text failures may reflect serving conditions rather than intrinsic model capability.

Protocol comparability. The content contract was shared, but API transports differed. Claude received distinct system and user fields while compatible routes received the supported combined transport. DeepSeek and GLM used low reasoning effort. The common 2,000-token limit may disadvantage routes that spend more output budget before emitting the requested JSON.

Statistical precision. Three repetitions expose stochastic variation but remain a small sample of future responses. Bootstrap and permutation results describe the observed task-repetition portfolio; they do not remove dependence among generated variants or guarantee stability after provider changes. Pairwise capability tests omit cells with route failures, while all-call metrics retain them.

Grading and execution. Ground truth and graders are tested, but any reference implementation defect would propagate systematically. Restricted subprocess execution is not container isolation. Confidence calibration is conditional on reported confidence, and the unsupported-issue proxy checks issue codes rather than every unsupported statement in free-form explanations.

12 Conclusion

ActuarialBench answers its central question with a qualified result. Claude and GPT formed the leading group under this protocol: Claude achieved the highest capability partial-credit mean, while GPT produced the most exact-correct calls and no route failures. Their mean capability difference was not statistically supported after correction. Both showed clearer paired advantages over DeepSeek and GLM, although performance varied materially by domain, task format, and operational outcome.

The experiment also identifies where the benchmark itself must improve. Validation tasks were too easy to distinguish the routes, provider cost was unavailable, and generated variants provide depth within families rather than broad coverage of actuarial practice. The defensible contribution is therefore not a universal leaderboard. It is a reproducible record of what each route solved, where it failed, how stable the result was, and which observed differences survived paired uncertainty analysis.

A Reproducibility Checklist

1. Create a Python 3.11 or later environment and install `pip install -e "[dev]"`.
2. Run `pytest` to verify task generators, graders, provider transports, retries, analysis, and report assets.
3. Set route keys through the environment variables documented in `.env.example`; never place keys in source files.
4. Run `python run_benchmark.py --smoke` for the bounded eight-task route check.
5. Run `python run_benchmark.py --repetitions 3 --parallel-models` only when the full API cost and scope are authorized.
6. Analyze an immutable experiment with `python analyze_results.py results/raw/EXPERIMENT`.
7. Generate report assets with `python generate_report_assets.py results/raw/EXPERIMENT`.
8. Compile this report twice with `pdflatex -interaction=nonstopmode -halt-on-error main.tex`.

The raw experiment contains 576 response records and 576 score records. The manifest freezes 48 task hashes, the shared prompt hash, route configuration, seed, repetitions, and execution commit. Generated tables and figures read from `analysis.json`; benchmark scores are not typed manually into this document.

B Artifact Map

Manifest	Experiment configuration, model routes, task and prompt hashes, seeds, timestamp, and execution commit.
Responses	Raw text, parsed output, provider metadata, latency, tokens, cost if reported, and error state for every call.
Scores	Objective score, exact-correct flag, component scores, confidence, schema status, and failure tags.
Analysis	Per-route capability, reliability, calibration, domain, task-format, and paired statistical results.
Report assets	Deterministic LaTeX fragments and figures generated from the frozen analysis file.